SECTION 1 · SUBSECTION 1.2
VEXFLOW ANALYSIS AND CLASS MAPPING

1.2.1 INTRODUCTION.

The engine copies VexFlow's mechanism wherever the problem is the same, and follows the printed Bhatkhande books wherever it differs. Four ideas carry over: elements report their widths before any x is assigned; contexts resolve collisions; the formatter owns x and the stave owns y; spanners are drawn last, from final positions (figure 1.2-1).

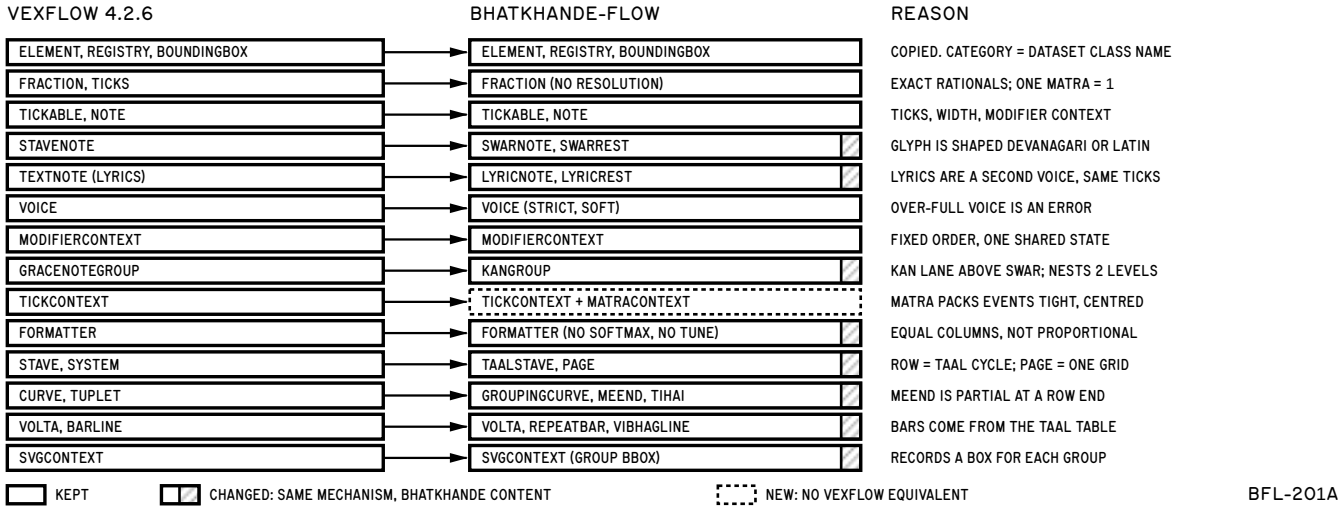


Figure 1.2-1. VexFlow Class Mapping

1.2.2 FUNCTIONAL DESCRIPTION.

Notes and modifiers join one ModifierContext per onset, then one TickContext per onset across the swar and lyric voices. A MatraContext groups the TickContexts of one matra, packs them tightly and centres the cluster in its column. TaalStave stacks the lanes; Page gives every column its x from one grid (figure 1.2-2).

1.2.2.1 Stacking Order.

preFormat() calls each modifier class once, inner to outer. All five share one state, so a class stacks outside what came before it: komal bar below the glyph, then mandra dots; tivra stroke above, then taar dots, then the kan lane.

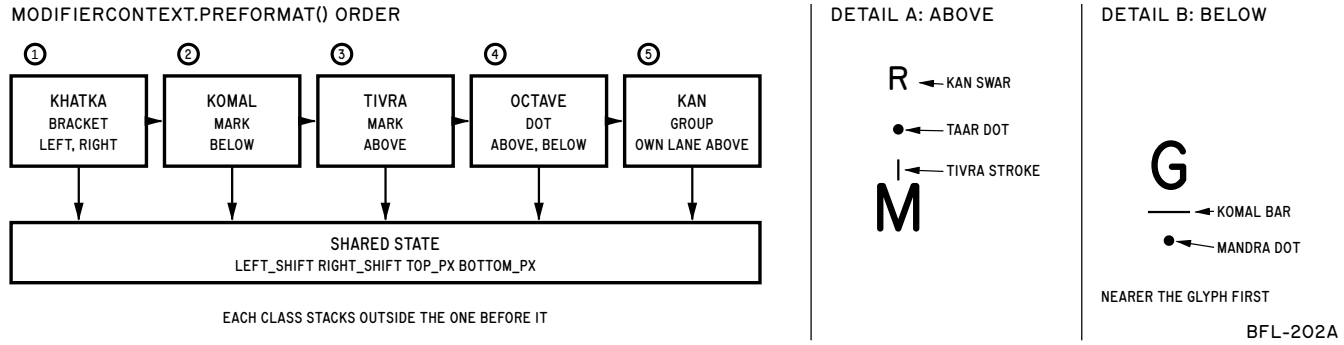


Figure 1.2-2. Modifier Stacking Order And Detail

1.2.3 PERFORMANCE AND DESIGN DATA.

Table 1.2-1. Corrections To The First Engine

Defect in v0.x	Rule in v1
Taal markers above the swars	BELOW THE LYRICS
Held swar written as avagraha	DASH IN SWAR LINE
Bar at the start of a row	NO OPENING BAR
Rows broken by page width	ROW = ONE TAAL CYCLE
Sub-beats spread apart	PACKED, CURVE UNDER
Matra width varies by row	ONE WIDTH PER PAGE
Kan as a small superscript	OWN LANE ABOVE SWAR
Parentheses read as murki	KHATKA BRACKETS
Grouping drawn as a bracket	CURVE UNDER SWARS

NOTE

Copy the mechanism when the problem is the same. When Bhatkhande differs, the scanned page decides, not Western habit.

ADD A MODIFIER — DEVELOPER

```
1 src/modifiers - new class
2 CATEGORY - dataset class name
3 static format(members, state)
4 ModifierContext - call in order
5 builder.decorate - attach it
6 labels.ts - add to classes
7 test - add a case
```

BFL